CO Assignment 2

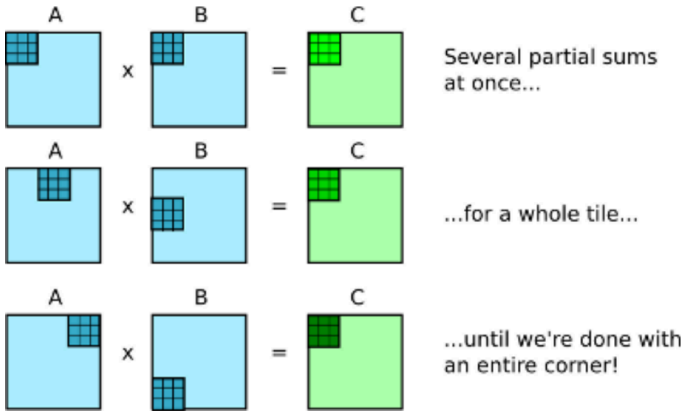
Matrix Multiplication

Name: Arnav Singhal
MIS: 612203012
SY B. Tech. Comp. Div 1

Problem Statement:

Understand the performance implications of operations on matrices, with row-major order and column-major order methods of accessing the matrix elements.

The given matrix_mul.c code multiplies two matrices. The naïve version of the matrix multiply is already written. The below image shows how a matrix can be tiled into smaller sub-matrices and then multiplied together to generate partial sums.



Using the logic of tiling illustrated in the above picture, write the tiled version of the matrix multiplication in the corresponding section indicated in matrix_mul.c code.

Assume that the matrix sizes (rows and columns) are powers of two and the tile sizes are also powers of two. Report the performance in terms of wall clock time for different tile values (2, 4, 8, 16 so on) for matrix sizes of 1024, 2048 and 4096. Provide the reason for performance improvement with tiling.

Challenge question: Use Linux perf tool to measure the key performance indicators such as cache accesses and hit rate and report these values for the above.

Matrix Multiplication - Naïve Version:

<u>Method 1 - Row wise implementation</u>

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)malloc(SIZE * sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void multiply_matrices(int **A, int **B, int **C) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            C[i][j] = 0;
            for (int k = 0; k < SIZE; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        free(matrix[i]);
    free(matrix);
}

int main() {
```

```
    int **A = create_matrix();
    int **B = create_matrix();
    int **C = create_matrix();

    init_matrix(A);
    init_matrix(B);

    multiply_matrices(A, B, C);

    printf("Matrix multiplication completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

Output and Performance:

```
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_multiply_row.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a' (5 runs):

          1,147.47 msec task-clock                #    0.989 CPUs utilized            ( +-  1.00% )
    3,98,36,31,635      cycles                    #    3.436 GHz                      ( +-  0.90% )
    8,77,34,47,284      instructions              #    2.19  insn per cycle           ( +-  0.00% )
      19,18,26,654      cache-references          #  165.458 M/sec                    ( +-  4.16% )
         65,78,781      cache-misses              #    3.276 % of all cache refs      ( +- 10.89% )
             3,137      page-faults               #    2.706 K/sec                    ( +-  0.02% )

            1.1599 +- 0.0115 seconds time elapsed  ( +-  0.99% )
```

Method 2 - Column wise implementation

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)malloc(SIZE * sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void multiply_matrices(int **A, int **B, int **C) {
    for(int j = 0; j < SIZE; j++) {
        for(int i = 0; i < SIZE; i++) {
            C[i][j] = 0;
            for(int k = 0; k < SIZE; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        free(matrix[i]);
    free(matrix);
}

int main() {
    int **A = create_matrix();
    int **B = create_matrix();
```

```
    int **C = create_matrix();

    init_matrix(A);
    init_matrix(B);

    multiply_matrices(A, B, C);

    printf("Matrix multiplication completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

Output and Performance:

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_multiply_col.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a' (5 runs):

          1,040.15 msec task-clock                #    0.996 CPUs utilized            ( +-  0.23% )
    3,61,65,76,716      cycles                    #    3.465 GHz                      ( +-  0.19% )
    8,77,52,51,072      instructions              #    2.42  insn per cycle           ( +-  0.00% )
      22,48,68,358      cache-references          #  215.414 M/sec                    ( +-  1.58% )
       1,30,61,542      cache-misses              #    5.688 % of all cache refs      ( +-  7.12% )
             3,139      page-faults               #    3.007 K/sec                    ( +-  0.01% )

           1.04424 +- 0.00237 seconds time elapsed  ( +-  0.23% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Matrix Multiplication - Optimized (using Tiling)

<u>Method 1 - Row wise implementation</u>

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)calloc(SIZE, sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void multiply_matrices(int **A, int **B, int **C, int tile_size) {
    for(int i = 0; i < SIZE; i += tile_size) {
        for(int j = 0; j < SIZE; j += tile_size) {
            for(int k = 0; k < SIZE; k += tile_size) {
                for(int ii = i; ii < i + tile_size && ii < SIZE; ii++) {
                    for(int jj = j; jj < j + tile_size && jj < SIZE; jj++) {
                        for(int kk = k; kk < k + tile_size && kk < SIZE; kk++) {
                            C[ii][jj] += A[ii][kk] * B[kk][jj];
                        }
                    }
                }
            }
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
```

```
        free(matrix[i]);
    free(matrix);
}

int main(int argc, char **argv) {
    int **A = create_matrix();
    int **B = create_matrix();
    int **C = create_matrix();

    init_matrix(A);
    init_matrix(B);

    multiply_matrices(A, B, C, atoi(argv[1]));

    printf("Matrix multiplication completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

<u>Output 1 - Tile size = 16</u>

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_mul_row_optimized.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 16
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 16
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 16' (5 runs):

            863.89 msec task-clock                #    0.998 CPUs utilized            ( +-  0.08% )
    2,99,80,65,252      cycles                    #    3.466 GHz                      ( +-  0.06% )
   12,86,12,55,629      instructions              #    4.28  insn per cycle           ( +-  0.00% )
      17,25,85,595      cache-references          #  199.532 M/sec                    ( +-  1.02% )
         26,99,463      cache-misses              #    1.546 % of all cache refs      ( +-  6.16% )
             3,140      page-faults               #    3.630 K/sec                    ( +-  0.01% )

          0.865358 +- 0.000703 seconds time elapsed  ( +-  0.08% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 2 - Tile size = 32

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 32
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 32
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 32' (5 runs):

           798.11 msec task-clock                #    0.995 CPUs utilized            ( +-  0.43% )
    2,76,85,82,672      cycles                    #    3.452 GHz                      ( +-  0.15% )
   12,40,57,90,923      instructions              #    4.48  insn per cycle           ( +-  0.00% )
       6,20,96,578      cache-references          #   77.434 M/sec                    ( +-  1.51% )
          26,92,484      cache-misses              #    4.320 % of all cache refs      ( +- 25.25% )
              3,140      page-faults               #    3.916 K/sec                    ( +-  0.02% )

          0.80243 +- 0.00349 seconds time elapsed  ( +-  0.43% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 3 - Tile size = 64

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 64
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 64
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 64' (5 runs):

           784.59 msec task-clock                #    0.992 CPUs utilized            ( +-  0.67% )
    2,72,70,92,831      cycles                    #    3.449 GHz                      ( +-  0.04% )
   12,19,14,13,345      instructions              #    4.47  insn per cycle           ( +-  0.00% )
       4,09,95,813      cache-references          #   51.850 M/sec                    ( +-  1.85% )
          20,01,004      cache-misses              #    4.724 % of all cache refs      ( +-  3.64% )
              3,142      page-faults               #    3.974 K/sec                    ( +-  0.02% )

          0.79112 +- 0.00540 seconds time elapsed  ( +-  0.68% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 4 - Tile size = 128

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 128
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 128
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 128' (5 runs):

            890.14 msec task-clock                #    0.995 CPUs utilized            ( +-  0.27% )
    3,08,53,84,226      cycles                    #    3.453 GHz                      ( +-  0.04% )
   12,08,77,80,840      instructions              #    3.91  insn per cycle           ( +-  0.00% )
       5,20,65,373      cache-references          #   58.262 M/sec                    ( +-  0.70% )
         18,41,516      cache-misses              #    3.490 % of all cache refs      ( +-  6.66% )
             3,141      page-faults               #    3.515 K/sec                    ( +-  0.02% )

           0.89418 +- 0.00237 seconds time elapsed  ( +-  0.26% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Method 2 - Column wise implementation

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)calloc(SIZE, sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void multiply_matrices(int **A, int **B, int **C, int tile_size) {
    for(int i = 0; i < SIZE; i += tile_size) {
        for(int j = 0; j < SIZE; j += tile_size) {
            for(int k = 0; k < SIZE; k += tile_size) {
                for(int jj = j; jj < j + tile_size && jj < SIZE; jj++) {
                    for(int ii = i; ii < i + tile_size && ii < SIZE; ii++) {
                        for(int kk = k; kk < k + tile_size && kk < SIZE; kk++) {
                            C[ii][jj] += A[ii][kk] * B[kk][jj];
                        }
                    }
                }
            }
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        free(matrix[i]);
    free(matrix);
```

```
}

int main(int argc, char **argv) {
    int **A = create_matrix();
    int **B = create_matrix();
    int **C = create_matrix();

    init_matrix(A);
    init_matrix(B);

    multiply_matrices(A, B, C, atoi(argv[1]));

    printf("Matrix multiplication completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

<u>Output 1 - Tile size = 16</u>

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_mul_col_optimized.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 16
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 16
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 16' (5 runs):

            896.09 msec task-clock                #    1.031 CPUs utilized            ( +-  0.93% )
    2,98,11,62,142      cycles                    #    3.432 GHz                      ( +-  0.17% )
   12,84,44,04,628      instructions              #    4.33  insn per cycle           ( +-  0.00% )
      14,13,22,739      cache-references          #  162.695 M/sec                    ( +-  0.63% )
       1,53,05,515      cache-misses              #   10.880 % of all cache refs      ( +- 16.07% )
             3,138      page-faults               #    3.613 K/sec                    ( +-  0.02% )

           0.86907 +- 0.00842 seconds time elapsed  ( +-  0.97% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 2 - Tile size = 32

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 32
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 32
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 32' (5 runs):

           855.90 msec task-clock                #    1.023 CPUs utilized            ( +-  0.81% )
    2,86,13,79,181      cycles                    #    3.424 GHz                      ( +-  0.14% )
   12,40,29,61,962      instructions              #    4.36  insn per cycle           ( +-  0.00% )
       7,21,97,863      cache-references          #   86.395 M/sec                    ( +-  1.91% )
       1,28,50,893      cache-misses              #   18.148 % of all cache refs      ( +- 15.55% )
             3,140      page-faults               #    3.757 K/sec

          0.83640 +- 0.00716 seconds time elapsed  ( +-  0.86% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 3 - Tile size = 64

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 64
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 64
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 64' (5 runs):

           881.81 msec task-clock                #    1.011 CPUs utilized            ( +-  1.07% )
    2,95,20,30,092      cycles                    #    3.387 GHz                      ( +-  0.20% )
   12,19,07,74,138      instructions              #    4.13  insn per cycle           ( +-  0.00% )
       6,48,63,918      cache-references          #   74.425 M/sec                    ( +-  2.64% )
          56,57,401      cache-misses              #    8.297 % of all cache refs      ( +- 16.58% )
             3,141      page-faults               #    3.604 K/sec                    ( +-  0.02% )

          0.87208 +- 0.00947 seconds time elapsed  ( +-  1.09% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Output 4 - Tile size = 128

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ ./a 128
Matrix multiplication completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e task-clock,cycles,instructions,cache-references,cache-miss
es,page-faults -r 5 ./a 128
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.
Matrix multiplication completed.

 Performance counter stats for './a 128' (5 runs):

           914.29 msec task-clock                #    0.984 CPUs utilized            ( +-  0.86% )
    3,15,39,50,382      cycles                    #    3.398 GHz                      ( +-  0.17% )
   12,08,74,36,235      instructions              #    3.82  insn per cycle           ( +-  0.00% )
       9,14,50,489      cache-references          #   98.514 M/sec                    ( +-  1.02% )
          15,30,608      cache-misses              #    1.653 % of all cache refs      ( +- 16.61% )
              3,141      page-faults               #    3.384 K/sec                    ( +-  0.01% )

          0.92869 +- 0.00791 seconds time elapsed  ( +-  0.85% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

Performance Overview:

Approach	Tile size	Execution time in ms (task-clock)	Clock cycles	Instructions	Cache accesses	Cache misses	Hit rate (%)	Page faults
Naïve (row wise)	-	1,147.47	3,98,36,31,635	8,77,34,47,284	19,18,26,654	65,78,781	96.57	3,137
Naïve (column wise)	-	1,040.15	3,61,65,76,716	8,77,52,51,072	22,48,68,358	1,30,61,542	94.19	3,139
Tiling (row wise)	16	863.89	2,99,80,65,252	12,86,12,55,629	17,25,85,595	26,99,463	98.44	3,140
	32	798.11	2,76,85,82,672	12,40,57,90,923	6,20,96,578	26,92,484	95.66	3,140
	64	784.59	2,72,70,92,831	12,19,14,13,345	4,09,95,813	20,01,004	95.12	3,142
	128	890.14	3,08,53,84,226	12,08,77,80,840	5,20,65,373	18,41,516	96.46	3,141
Tiling (column wise)	16	896.09	2,98,11,62,142	12,84,44,04,628	14,13,22,739	1,53,05,515	89.17	3,138
	32	855.90	2,86,13,79,181	12,40,29,61,962	7,21,97,863	1,28,50,893	82.2	3,140
	64	881.81	2,95,20,30,092	12,19,07,74,138	6,48,63,918	56,57,401	91.28	3,141
	128	914.29	3,15,39,50,382	12,08,74,36,235	9,14,50,489	15,30,608	98.33	3,141

CONCLUSION:

1. **Tiling Improves Cache Efficiency**

- The blocked (tiled) approach used in the optimized implementation for matrix multiplication improves cache locality by working on smaller submatrices that fit within the cache.
- This reduces cache misses, leading to better performance compared to a naive row-wise or column-wise approach.

2. **Row-wise vs. Column-wise Multiplication**

- The programs clearly demonstrate that row-wise matrix multiplication is generally more efficient than column-wise multiplication.
- This is due to the better cache performance and memory access patterns associated with row-wise multiplication.
- The programs highlight the impact of memory access patterns on computational performance.

3. Tile Size Selection Affects Performance

- The choice of `tile_size` has a direct impact on performance.
- Too small: More overhead due to loop control
- Too large: Cache inefficiencies due to excessive evictions
- Tuning `tile_size` based on L1/L2 cache sizes is crucial for optimal performance.

4. Performance Analysis

- An in-depth performance analysis can be done using perf, including profiling CPU usage, memory access patterns and hardware counters.
- It reveals details about instruction counts, cache misses and other performance-critical metrics, aiding in performance optimization.

5. Hardware Impact

- The performance of the programs and the insights provided by ‘perf’ can be influenced by the specific hardware and cache architecture of the machine on which they are executed.
- Different hardware configurations may yield varying performance results. This was seen in the case of the naive approach for matrix multiplication, where row wise multiplication is typically more efficient than column wise multiplication. Here, the opposite of this was observed.

6. Real World Application

- While the programs focus on matrix multiplication as a simplified example, the lessons learned can be applied to real-world applications where efficient memory access patterns are essential for achieving optimal performance.
- In summary, the matrix multiplication programs, and the use of the ‘perf’ tool illustrate the importance of memory access patterns, cache optimization, and performance analysis in software development.
- They showcase how careful consideration of these factors can lead to more efficient code execution and better overall program performance.